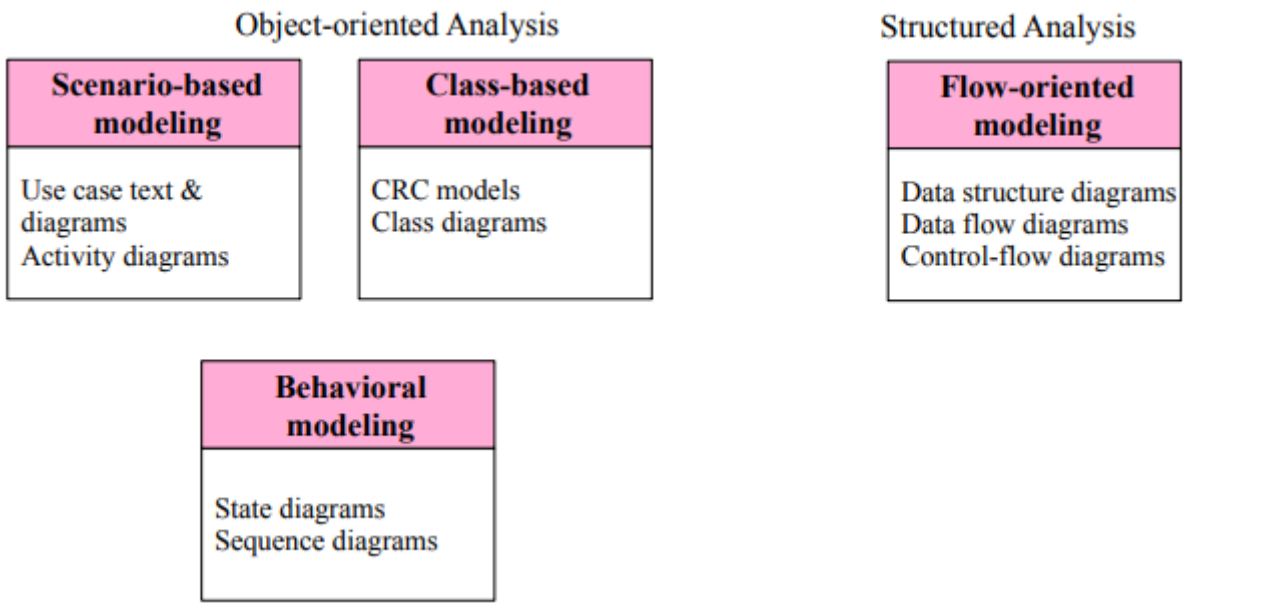
Lecture-4

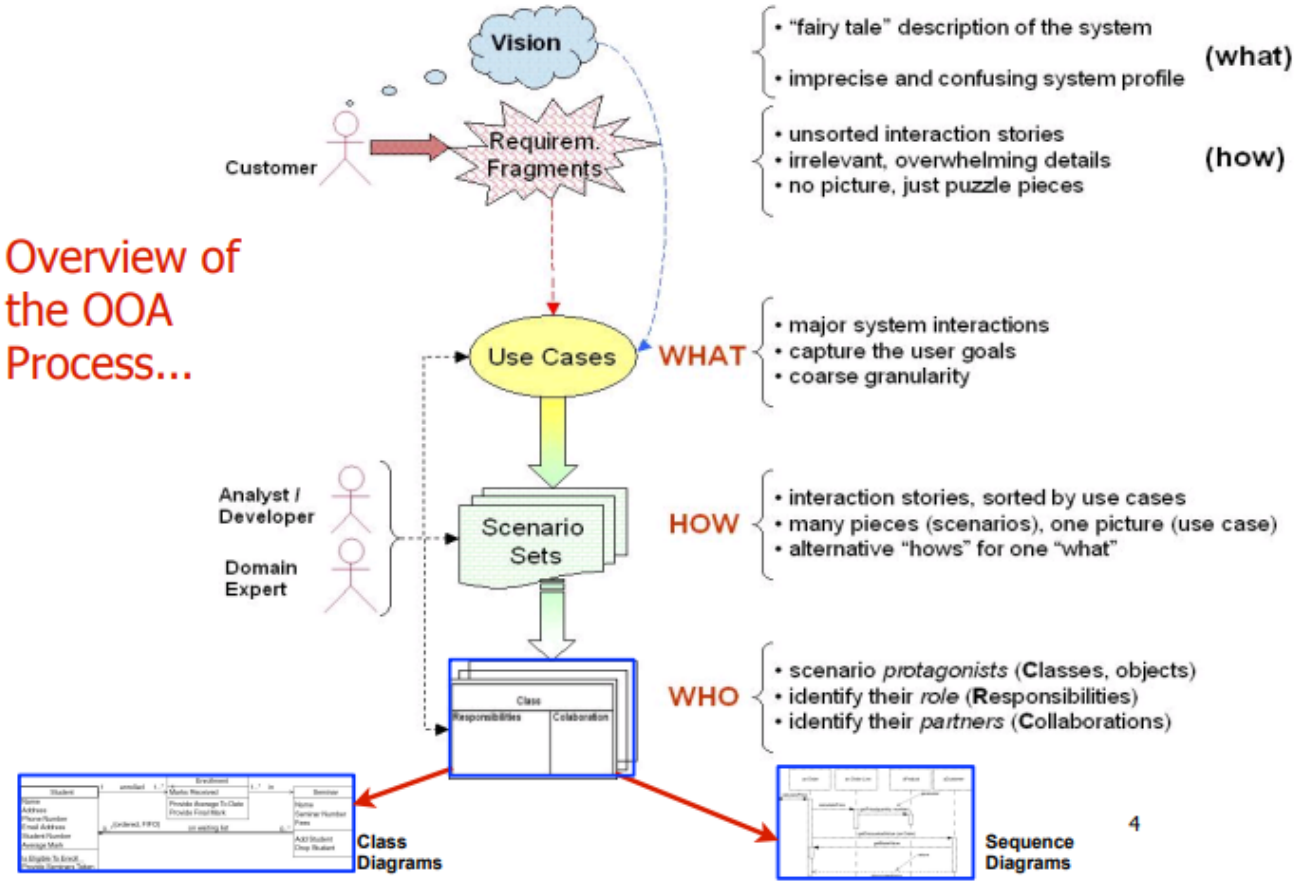
Object-Oriented Analysis and Modeling

Object-Oriented Analysis (OOA)

- What are the relevant objects? How do they relate to one another?
- How do we specify/model a problem so that we can create an effective design?
- OOA aims to model the problem domain, by developing an object-oriented (OO) system.

Elements of the Analysis Model





➢ Avem usecase-urile care contin o multime de scenarii

➢ Scenariile sunt formate din : clase si obiecte, metodele claselor si colaboratorii care reprezinta alte clase -> ele interactioneaza

➢ Pt fiecare scenariu avem cate o diagrama de secventa si o diagrama de clasa

➢ Diagr. De secventa sau un set de diagrame de secventa -> putem construi o diagram de clase din ce in ce mai bogata

Class-Responsibility-Collaborator (CRC) Modelling

- Technique to identify candidate classes and indicate their responsibilities and collaborators
  - K.Beck&W.Cunningham (1989), R.Wirfs-Brock(1990,2002)
- Uses simple index cards

Class	
Responsibilities	Colaborators

Class name:	
Class type: (e.g., device, property, role, event)	
Class characteristic: (e.g., tangible, atomic, concurrent)	
responsibilities:	collaborations:

CRC ne imparte clasa in responsabilitati si colaboratori. Responsabilitatile -> metodele

```
➢ Asta de mai sus reprezinta ideea de CRC cards -> ne ajuta sa identificam
clasele si obiectele
➢ Cum gasim clasele? Ascundem datele si orferim servicii
➢ Am captat cerintele -> aflam care sunt usecase-urile -> am modelat
clasele -> am facut relatia dintre ele -> am gasit digrama de clasa si de
secventa -> nu mergem mai in detaliu
```

CRC Cards Session Scenario

- Use-Case driven
  - a use-case is the token
- Goal: be able to go through the whole use-case description by using the responsibilities written on the CRC cards
  - ...and of course following the Collaborator links

-> goal ul este practic sa ne formam clasele din use case uri

Rules for Identifying Classes

1. Retained Information
   - information about object must be remembered for the system to function
2. Needed Services
   - have a set of operations that change the value of its attributes
3. Multiple Attributes
   - focus on “major” information
   - object with single attribute is ok during design, but during analysis is just an attribute of another object
4. Essential Requirements objects
   - entities that produce or consume information of the system, in any solution

1. Sa ai unde sa stochezi informatia

```
**Explicație detaliată:**
- Într-un sistem software, există obiecte care trebuie să păstreze anumite informații pe termen lung, pentru ca sistemul să funcționeze corect
- Aceste obiecte dețin "starea" sistemului – adică date esențiale care nu pot fi calculate la nevoie, ci trebuie stocate.
- De exemplu:
- Într-un sistem bancar: contul clientului → număr cont, sold curent, istoricul tranzacțiilor.
Într-un sistem de gestiune a comenzilor: comanda → data comenzii, lista produselor, statusul livrării.
- Dacă aceste informații nu ar fi memorate, sistemul ar pierde contextul necesar funcționării corecte.
```

   Pe scurt: Obiectele cu "retained information" sunt acelea care păstrează starea sistemului între operații.
2. Practic sa ti faci setterele

   Explicație detaliată:

- Orice obiect dintr-un sistem nu este doar un "depozit de date", ci oferă și servicii (operații/metode) care modifică aceste date.
- Serviciile sunt acele acțiuni pe care le poate executa obiectul asupra propriilor atribute.
- De exemplu:
  - Cont bancar → `depunere()`, `retragere()`, `transfer()`.
  - Comandă online → `adaugăProdus()`, `confirmăComanda()`, `anulează Comanda()`.
- Aceste servicii definesc comportamentul obiectului și modul în care interacționează cu alte obiecte.

Pe scurt: Obiectul nu doar păstrează date, ci oferă și funcționalități concrete pentru modificarea sau prelucrarea acelor date.

3. E imp sa ai mai multe atribute intr-o singura clasa (daca ai unul singur, nici nu e neaparat nevoie sa ai nevoie de clasa)
   Explicație detaliată:

- În timpul analizei sistemului (prima etapă), ne concentrăm pe obiecte care dețin **mai multe atribute relevante**.
- Obiectele cu un singur atribut nu sunt în general considerate obiecte principale, ci mai degrabă părți componente ale altor obiecte.
- De exemplu:
  - Un obiect `Adresa` (cu strada, oraș, cod poștal) este valid.
  - Dar un obiect `CodPostal` (cu un singur câmp `cod`) ar putea fi mai potrivit ca atribut în `Adresa`, nu ca obiect de sine stătător.
- În proiectare (design), putem alege să creăm și obiecte simple (chiar cu un singur atribut) pentru claritate, reutilizare, sau design orientat pe micro-servicii.

Pe scurt: În analiza cerințelor ne uităm la entități complexe, cu mai multe informații relevante; obiectele simple tind să fie atribute în cadrul altor obiecte.

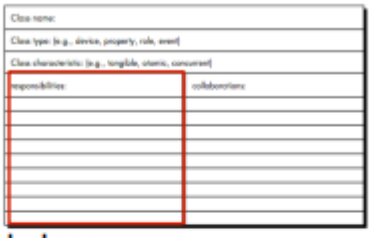
4. **Explicație detaliată:**

- Aceste obiecte sunt considerate **obiecte esențiale pentru cerințele sistemului**, deoarece:
  - Fie produc informații noi (ex: senzorul de temperatură produce date).
  - Fie consumă informații (ex: un raport de analiză care folosește datele colectate).
- Sunt obiecte care există indiferent de modul concret de implementare al sistemului.
- Ele reflectă o necesitate reală a sistemului, din punct de vedere funcțional.
- De exemplu:
  - Într-un sistem de gestiune medicală: `Pacient`, `Doctor`, `Programare`.
  - Într-un sistem e-commerce: `Client`, `Produs`, `Comandă`.

Pe scurt: Sunt acele obiecte care reflectă cerințele fundamentale ale sistemului, fiind necesare în orice implementare viitoare.

Identifying Responsibilities

Identifying Responsibilities



- attributes and operations of an identified class
- **Guidelines of Wirfs-Brock**:
  1. *System intelligence should be evenly distributed*
  2. *Information about one thing should be localized within a single class*
  3. *Information and its related behavior should stay in the same class*
  4. *Responsibilities should be shared among related classes*

-> Responsabilitatile sunt atribute(ce date retine) si operatii(metode)

1. Evităm centralizarea excesivă a logicii într-o singură clasă.
2. Grupăm datele care au legătură într-o singură clasă.
3. Clasa care deține datele trebuie să știe să le și manipuleze.
4. - Nicio clasă nu trebuie să știe mai mult decât este nevoie din responsabilitatea altora.
   Pe scurt: Colaborarea corectă între clase este cheia unui design curat.

Operations = Verbs

- **Computation**
- **Manipulation of data**
  - e.g., add, delete, modify attributes
- **Query**
  - about the state of an object
- **Monitor** an object
  - for the occurrence of a controlling event

- Has knowledge about the state of class and its associations

Collaborations

- Class can fulfill responsibilities by:
  1. using its own operations to manipulate its own attributes
  2. collaborating with others
- Three types of generic relationships:
  3. *has-knowledge-of (association)*
  4. *is-part-of (aggregation)*
  5. *composition*

Clasa isi poate modifica singura atributele prin propriile metode sau poate colabora cu alte clase

3️⃣ has-knowledge-of (association)

> *Are cunoștință despre...*

- Clasa A "știe de existența" clasei B.
- Este cea mai slabă formă de relație.
- Înseamnă doar că una dintre clase are o referință (pointer, obiect, id) către cealaltă.
- Nu implică dependență de viață (ciclul de viață al obiectelor este independent).

Exemplu:

- `Profesor` are o listă de `Student` (dar dacă un student dispare, profesorul poate continua să existe).

4️⃣ is-part-of (aggregation)

> *Este parte din... (dar poate exista separat)*

- Relație de tip "whole-part" (întreg - parte).
- Obiectele părți pot exista independent de obiectul întreg.
- Se folosește atunci când componenta poate exista și fără întreg.

Exemplu:

- `Echipă` agregă mai mulți `Jucători`. Jucătorii pot exista și fără echipă.
- În UML se desenează cu **romb gol**.

5️⃣ composition

> *Compoziție (parte dependentă complet de întreg)*

- Relație mult mai strânsă decât agregarea.
- Componentele nu pot exista fără întreg.
- Dacă întregul este distrus, automat și părțile sunt distruse.

Exemplu:

- `Mașină` compune `Motor`. Dacă ștergi obiectul `Mașină`, nu mai are sens ca motorul să existe.
- În UML se desenează cu **romb plin**.

Heuristics for Object-Oriented Modeling

Problema Proliferarii Claselor [Riel96]

- Spaghetti Code vs. Ravioli Code

- Cum se manifesta “Codul Ravioli”
  - Vreau sa adaug o facilitate noua in sistem. Care 23 de clase din cele 4.200 de clase trebuie sa le modific?

Fiti retinuti in a modela ca si clase
entitati din afara sistemului de implementat!

- Exemplu: Clientul unui Bancomat
  - trimite un mesaj bancomatului

Clase sunt acelea care PRIMESC MESAJE (sunt apelate)
nu cele care TRANSMIT MESAJE (apeleaza)!

!!! Clasele sunt cele care sunt apelate, nu cele care apeleaza

◆ Spaghetti Code

- Este binecunoscutul cod dezorganizat, greu de urmărit:
  - Logica e împrăștiată.
  - Nu există separare clară a responsabilităților.
  - Multe dependențe circulare.
  - Fără structuri clare de obiecte, metode, module.
- De obicei apare în sisteme monolitice, cu funcții uriașe care apelează altele fără o structură clară.

Imaginează-ți o farfurie cu spaghete: totul e încurcat, nu poți trage un fir fără să le tragi pe toate.

◆ Ravioli Code

- Apare adesea în sisteme OO (Object-Oriented) în care avem **prea multe clase**, fiecare foarte mică, foarte izolată.
- Fiecare clasă are puține atribute și metode, dar sistemul total ajunge la mii de clase.
- Problema apare atunci când vrei să faci o modificare:
  - Nu știi în câte clase trebuie să umbli.

    - O funcționalitate aparent simplă implică modificarea simultană în multe clase.
- Este "opusa" Spaghetti, dar tot problematic.

Imaginează-ți o farfurie cu ravioli: fiecare bucățică este separată, dar ca să găsești umplutura corectă trebuie să deschizi multe pachețele.

🔶 Exemplu din slide:

> „Vreau să adaug o facilitate nouă în sistem. Care 23 de clase din cele 4200 trebuie să le modific?”

- Adică sistemul este atât de fragmentat, încât orice mică schimbare implică modificări în zeci de clase.
- Apare o complexitate ascunsă: ai multe interfețe mici, dar modificările sunt greu de propagat.

🔶 Recomandare:

> "Fiți reținuți în a modela ca și clase entități din afara sistemului de implementat!"

- La analiza sistemului, modelăm numai clasele care *fac parte din sistemul nostru*, nu pe toate entitățile din lumea externă.
- Externii (clienți, utilizatori, furnizori etc.) vor interacționa cu sistemul, dar nu trebuie modelați neapărat ca obiecte în sistem.
- De exemplu: `ClientBancomat` poate fi un actor (în UML), dar nu neapărat o clasă.

🔶 Ce sunt clasele din punct de vedere al mesajelor:

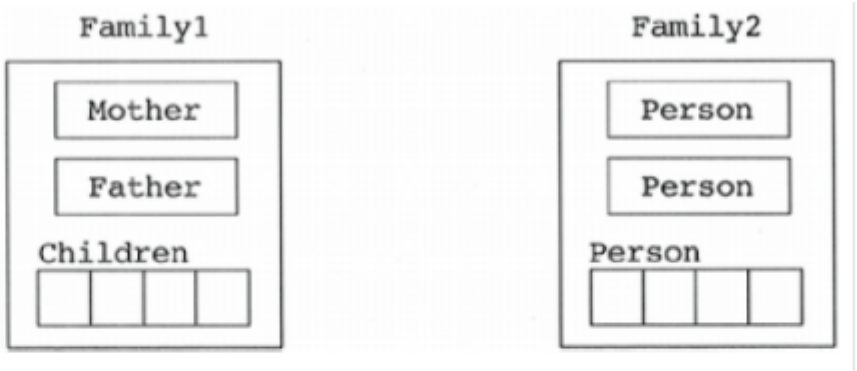
> *Clase sunt acelea care PRIMESC MESAJE (sunt apelate), nu cele care TRANSMIT MESAJE (apelează)*

- În analiza responsabilităților, definim clasele prin ceea ce li se cere să facă (prin ce mesaje/comenzi primesc).
- Practic:
    - Dacă un obiect este apelat de alții, atunci are o responsabilitate în sistem.
    - Dacă doar apelează alte obiecte și nu are logică proprie, poate nu e justificat ca obiect separat.

Exemplu:

- `Bancomat` primește mesaje: `retrageBani()`, `verificareSold()`, `depunereBani()`.
- `Client` transmite mesaje, dar în sistemul nostru este actor extern.

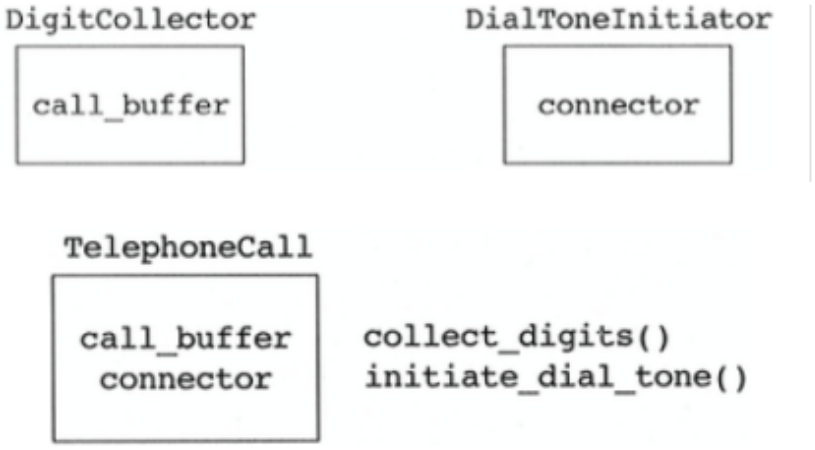
Roluri vs Clase



- Comportamentul este cel care decide!
  - daca comportamentul difera: **avem clase**;
  - daca nu: doar roluri ale aceleeasi clase
  - depinde de domeniul modelat de aplicatie
    - *naste()* vs. *schimbaScutece()*

Euristici pentru Eliminare Claselor Inutile

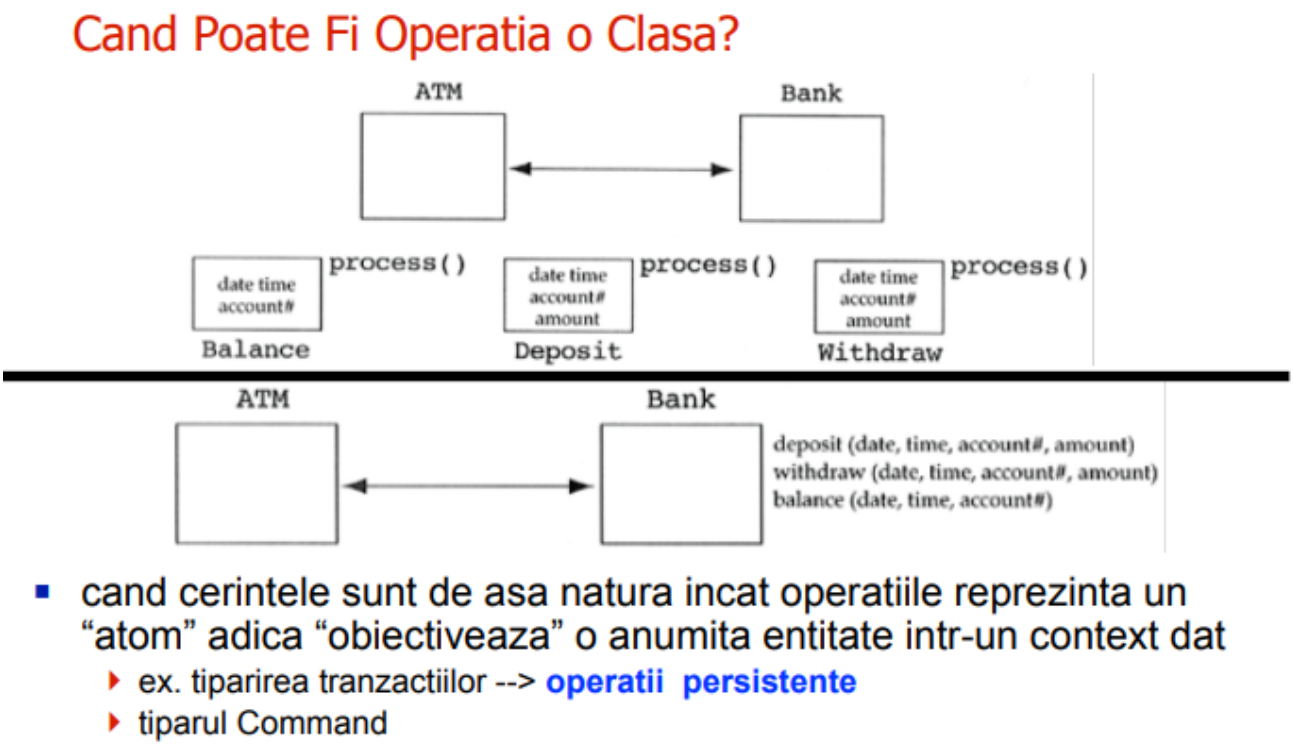
Nu transformati operatiile in clase




```
➢ Problema proliferarii claselor : programarea goto e foarte dificila ->
apare spaghetti code -> Solutia ar fi programarea structurata -> am inlocuit
spaghetti code cu ravioli code- > am niste bucati mici de paste, dar toata
farfuria e plina de paste -> control flow e foarte clar, stiu de unde pana
unde se duce, stiu cine apeleaza pe cine , dar nu stiu daca am o mie de
```

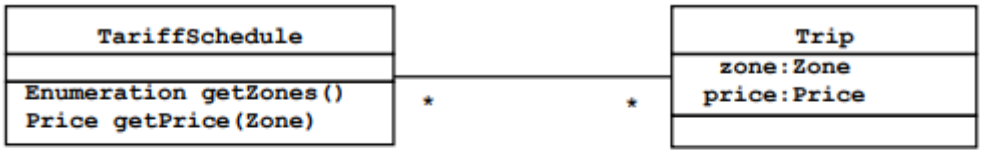
```
clase, pe care clase ar trebui sa le modific ca sa implementez eu stiu ce
feature -> te trezesti pierdut in miile de clase -> cum se poate Evita? Sa
evitam entitatile care sunt in afara sistemului (ne putem prinde din ex cu
calatorul -> calatorul e de fapt actorul aici)
➢ O clasa e cu adevarat clasa atunci cand pe ea o apeleaza alte clase ->
faptul ca ea apeleaza pe altele nu garanteaza ca e o clasa, dar daca e
apelata de alte clase atunci ea e sigur o clasa
➢ O alta situatie care ar putea duce la proliferarea claselor ar fi
tentatia de a modela fiecare element/substantiv ca fiind o clasa distincta o
➢ Am 4 clase : family, mother, father si child -> mother si father ar putea
fi scrise sub forma aceleiasi clase person/adult? Da -> elementul decisiv
care ne arata daca trebuie sa avem clase diferite sau nu este comportamentul
-> daca 2 obiecte ofera aceleasi servicii , atunci nu are sens sa modelez cu
2 clase distincte -> O clasa inseamna : am un comportament distinct pe care
altcineva nu il are sau nu poate fi executat de alta clasa
```

Cand poate fi Operatia o clasa?



- cand cerintele sunt de asa natura incat operatiile reprezinta un "atom" adica "obiectiveaza" o anumita entitate intr-un context dat
  - ex. tiparirea tranzactiilor --> **operatii persistente**
  - tiparul Command

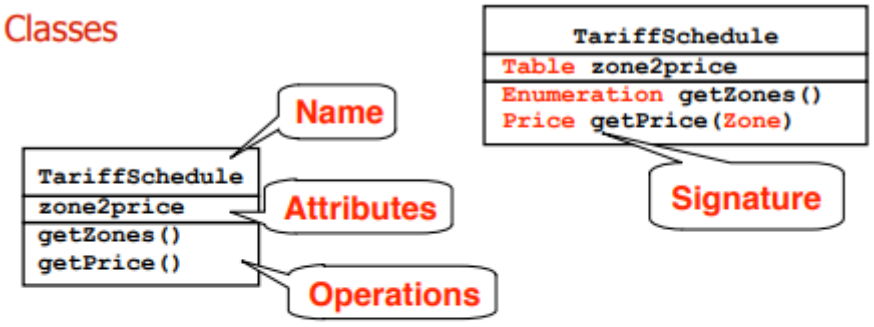
Class Diagrams



- Class diagrams represent the structure of the system.
- Class diagrams are used
  - during requirements analysis to model problem domain concepts
  - during system design to model subsystems and interfaces
  - during object design to model classes.

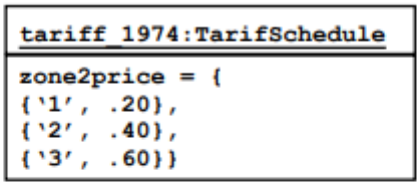
```
➢ Reprezinta potentialele relatii intre clase
➢ * inseamna ca prima clasa poate avea oricate obiecte de cea de care e conectat
```

Classes



- A class represents a concept.
- contians state (attributes) and behavior (operations).
- Each attribute has a type.
- Each operation has a signature.
- The class name is the only mandatory information.

Instances



- An **instance** represents a phenomenon.
- The name of an instance is <u>underlined</u> and can contain the class of the instance.
- The attributes are represented with their **values**.

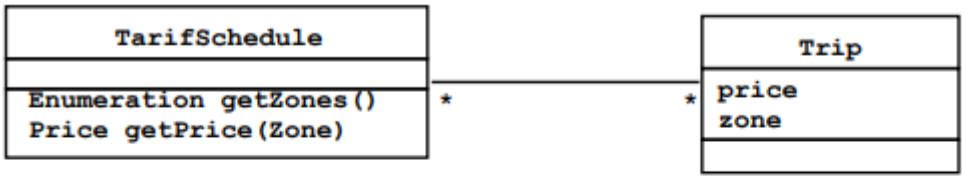
```
➢ Instanta unei clase = un obiect
```

Instance vs Class

De ce nu e o pipa? Pentru ca e o poza

```
➢ Obiectele pot fi reprezentate si ele -> obiectele sunt instante ale
claselor
➢ Un obiect e ceva real -> o clasa e reprezentarea oricarui obiect de tipul
clasei respective, nu e ceva real, palpabil
```

Associations



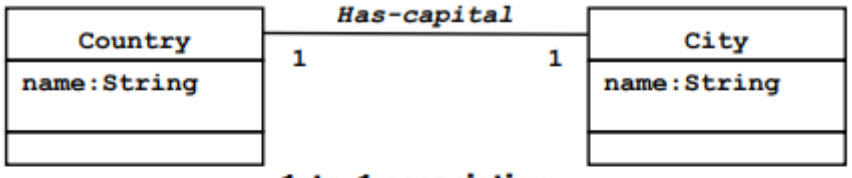
- Associations denote relationships between classes.
- The multiplicity of an association end denotes how many **objects** the source object can legitimately reference.

Deci daca avea steluta doar la TarifSchedule, atunci doar Trip ar fi putut contine oricate obiecte de tip TarifSchedule

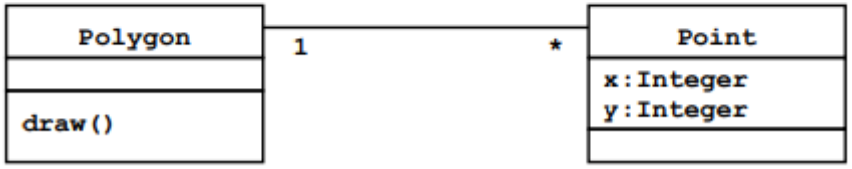
```
➢ Cand avem * -> un obiect TarifShedule poate avea oricate obiecte Trip ->
ma intereseaza multiplicitatea obiectelor, nu clasele
➢ Elem essential e nr de relatii posibile intre instantele acelor clase ,
relatia dintre obiectele acelor clase
```

```
➢ Elem essential e practice lucrurile care se vad cel mai putin -> astea se leaga cel mai tare de relatii
```

1-to-1 and 1-to-Many Associations



1-to-1 association

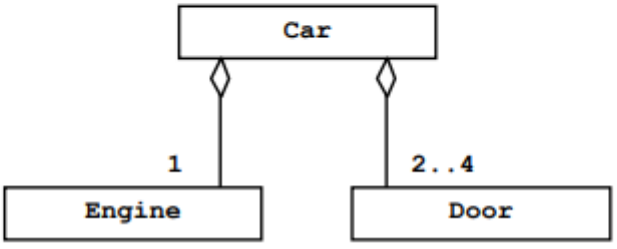


1-to-many association

```
➢ Relatie 1 la 1 : 1 obiect de tip tara e asociat cu 1 obiect de tip oras -> relatie speciala
➢ Relatie 1 la mai multi : 1 obiect de tip polygon e asociat cu mai multe obiecte de tip Punct -> * poate sa insemne 0 sau mai multe puncte -> spunem mai correct 1 obiect de tip polygon e asociat cu 3 sau mai multe obiecte tip punct ➢ O diagram banala -> conteaza enorm multiplicitatea relatiilor
```

Aggregation

- An **aggregation** is a special case of association denoting a "*consists of*" (HAS-A) hierarchy.
- The aggregate is the parent class, the components are the children class.

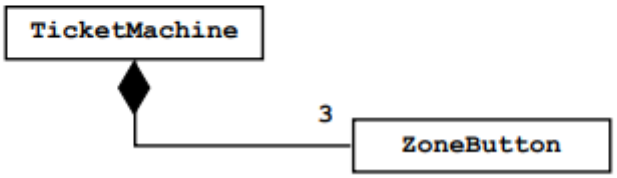



```
➢ Agregarea -> e ft importanta pt felul in care percepe lumea in general -> specifica clar cate obiecte se afla in relatia cu obiectele clasei parinte
 ➢ Ex: avem o clasa parinte masina si 2 subclase motor si usa -> avem relatie de agregare intre clasa masina si subclasa motor in care 1 obiect de tip masina poate avea 1 singur obiect de tip motor - > avem relatie de
```

```
agregare intre 1 obiect de tip masina si subclasa usa : 1 obiect de tip
masina poate avea intre 2 sau 4 obiecte tip usa si doar atat
```

Composition

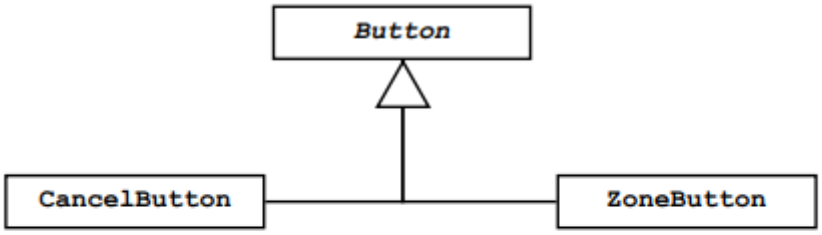
- A solid diamond denote **composition**, a strong form of aggregation where components cannot exist without the aggregate.




```
➢ Avem un caz si mai special care se numeste compozitie : ne da o forma mai
stricta a relatie de agregare
➢ Intrebare : obiectele motor si usa se construiesc special pt a construi
acest obiect masina si daca obiectul masina se distruge, se vor distruge si
ele? Sau ele sunt independente de obiectul masina? Ele vor exista si fara
masina
➢ Diferenta dintre compozitie si agregare : cand ai o relatie de agregare,
relatia e mai slaba, obiectele motor si usa nu trebuie sa fie special create
pentru a crea un obiect masina, ele pot exista inainte si pot fi folosite si
de masina asta noua, iar cand se distruge obiectul masina nu e neaparat sa
se distruga si aceste obiecte de tip motor si usa -> in schimb, la relatia
de compozitie e fix invers, cand se distruge obiectul ticketmachine automat
se distruge si obiectul zonebutton
➢ Asemanare : ambele functioneaza pe principiul “are un” , “ are o” sau
“are mai multe” -> la ambele nu poti sa construiesti obiectul intreg daca nu
am acel nr indicat in diagrama
    !! LA AMBELE TIPURI DE RELATII trebuie sa stiu cate obiecte are

Alt ex : Ai un hotel si poti sa faci rezervari. Daca se distruge hotelul, nu
mai poti sa faci rezervari (relatie de compozitie)
```

Generalization



- Generalization relationships denote inheritance between classes.
- The children classes inherit the attributes and operations of the parent class.
- Generalization simplifies the model by eliminating redundancy.

```
➢ Relatia de generalizare : mostenirea, dar nu in totalitate
➢ Cum reduc proliferarea claselor? Solutie : mai multe obiecte pot fi
instante ale aceleiasi clase
➢ Procesul de generalizare : pornim de la 2 clase oarecare pe care le gasim
cancelButton si zoneButton -> ele au butonul comun, deci se poate generaliza
sau se poate da factor comun si putem construi o clasa Button
➢ Generalizarea simplifica modelarea prin eliminarea redundantei
```

Sequence Diagrams

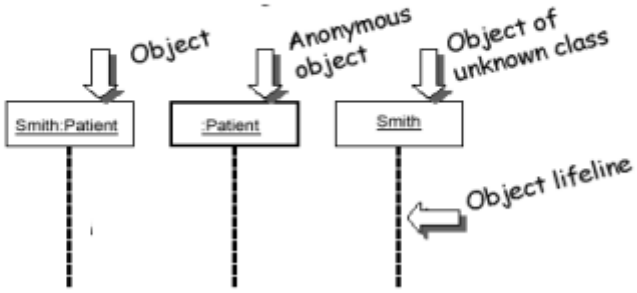
UML Sequence Diagrams

- Sequence Diagram: an "interaction diagram" that models a single scenario executing in the system
  - perhaps 2nd most used UML diagram (behind class diagram)
- Participant: an object or entity that acts in the sequence diagram
  - sequence diagram starts with an unattached "found message" arrow
- Message: communication between participant objects
- Axes in a sequence diagram:
  - horizontal: which object/participant is acting
  - vertical: time (down -> forward in time)

-> Interactiuni intre obiecte

Representing Objects

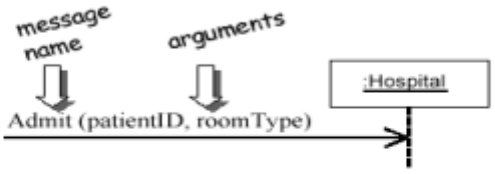
- squares with object type, optionally preceded by object name and colon
  - write object's name if it clarifies the diagram
  - object's "life line" represented by dashed vert. line



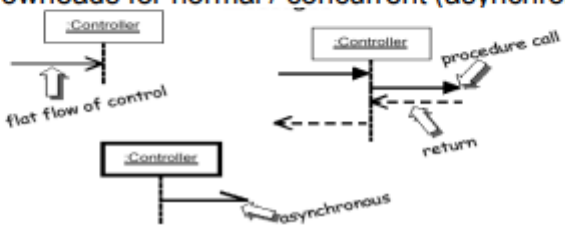
Name syntax: <objectname>:<classname>

Messages between objects

- message (method call) indicated by horizontal arrow to other object
  - write message name and arguments above arrow

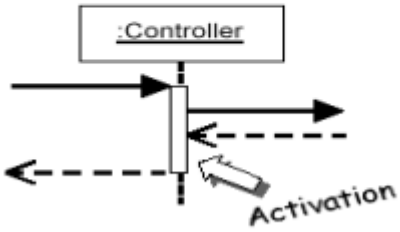


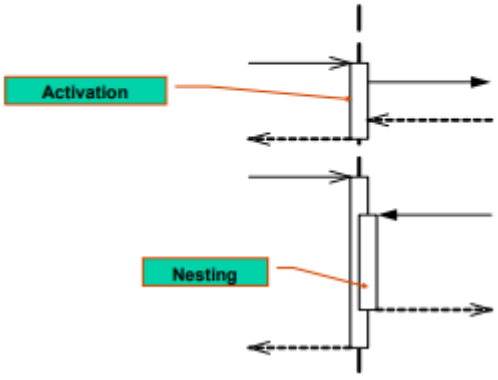
  - dashed arrow back indicates return
  - different arrowheads for normal / concurrent (asynchronous) methods



Indicating method calls

- Activation: shows when object's method is on the stack
  - either that object is running its code, or it is on the stack waiting for another object's method to finish
  - nest to indicate recursion

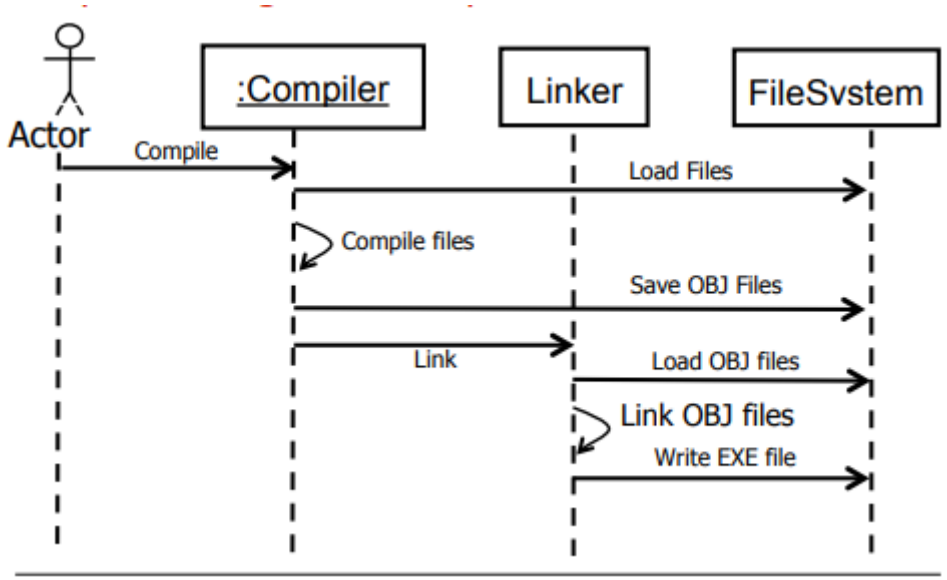




Example 1 :

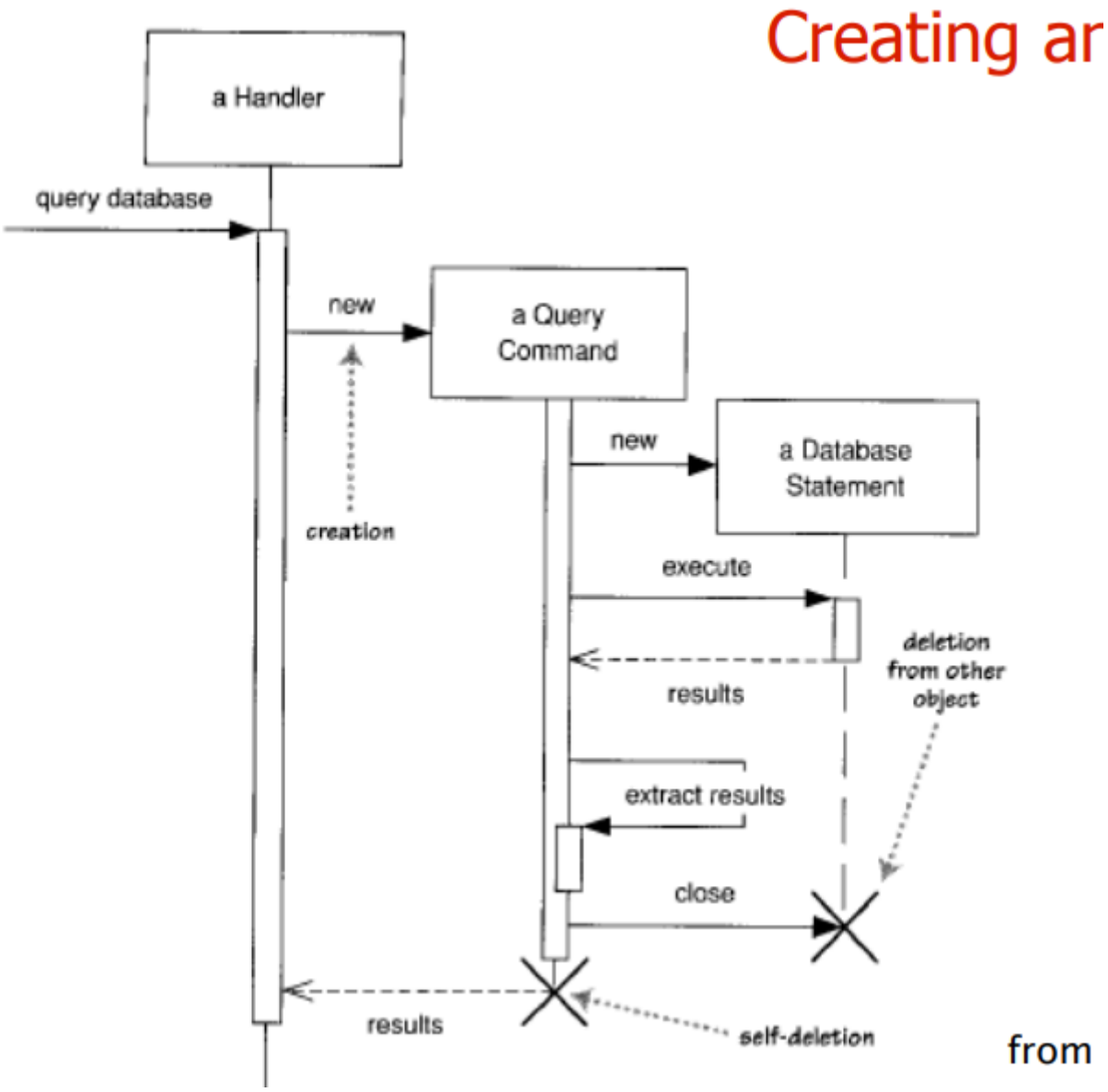
- Building an executable from sources
  - load source files and compile them
  - load resulting object files and link them
  - write executable file

Sequence Diagram – Compilation



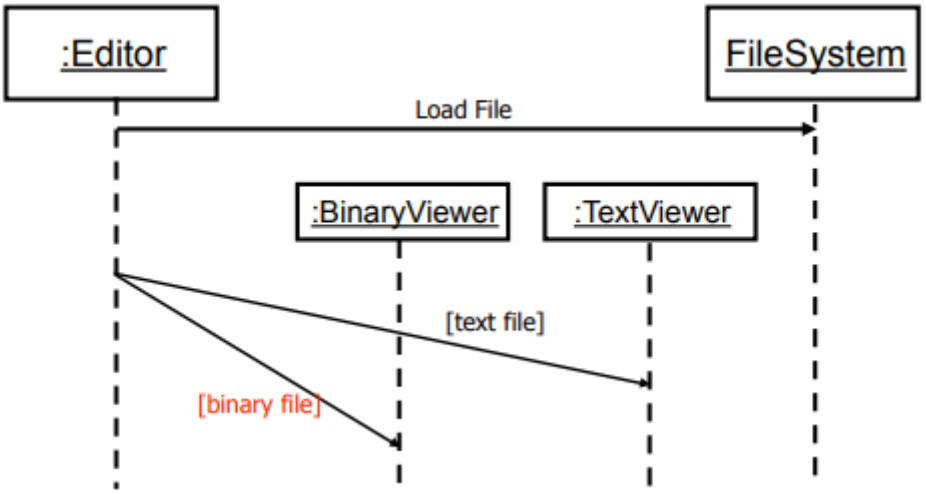

```
➢ Ce se intampla cand o metoda compile() se apeleaza pe un obiect Compiler?
Obiectul Compiler interactioneaza cu obiectul fileSystem . De ce? Ca sa
incarc toate fisierele sursa (se indica pe sageata)
➢ De unde stiu cand se termina un apel? Uneori, cand pe return se
construieste un obiect nou, cu care mai apoi voi interactiona -> e ft
important sa pui sageata de return ca sa stim ca de acolo ai obiectul ala
```

Creating and Deleting Objects

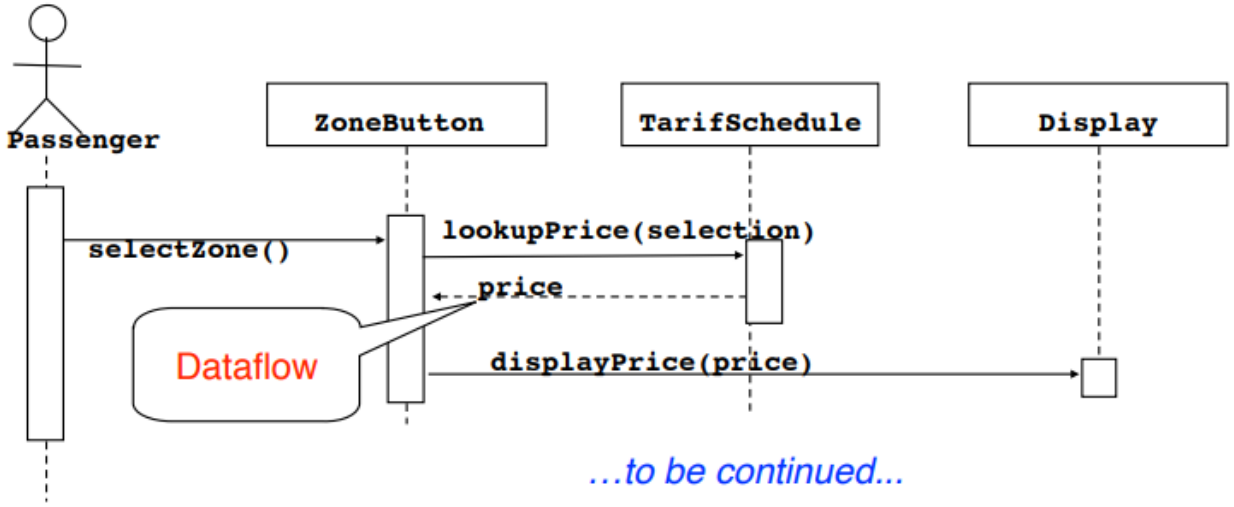



```
➢ Ce vrea sa ne arate diagram asta de secventa e ca un obiect poate exista
sau poate fi creat si mai tarziu in program, nu e neaparat la inceput, si se
si pot distruge mai repede
```

Branching Flow : flow goes to different objects [if condition is met]



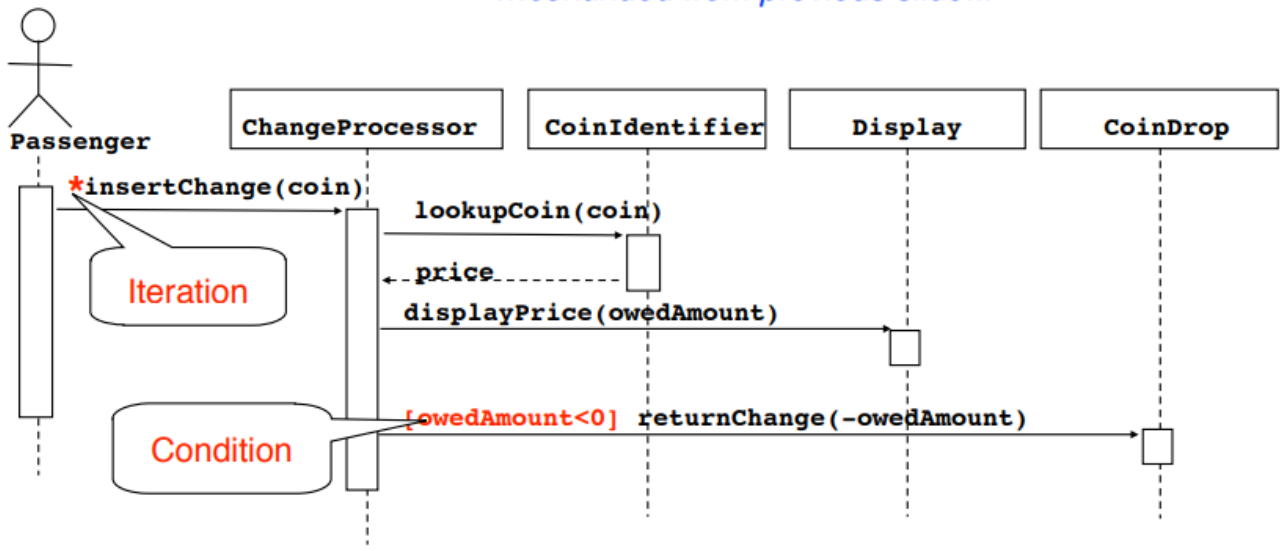
Flow of messages



- The source of an arrow indicates the activation which sent the message
- An activation is as long as all nested activations
- Horizontal dashed arrows indicate data flow
- Vertical dashed lines indicate lifelines

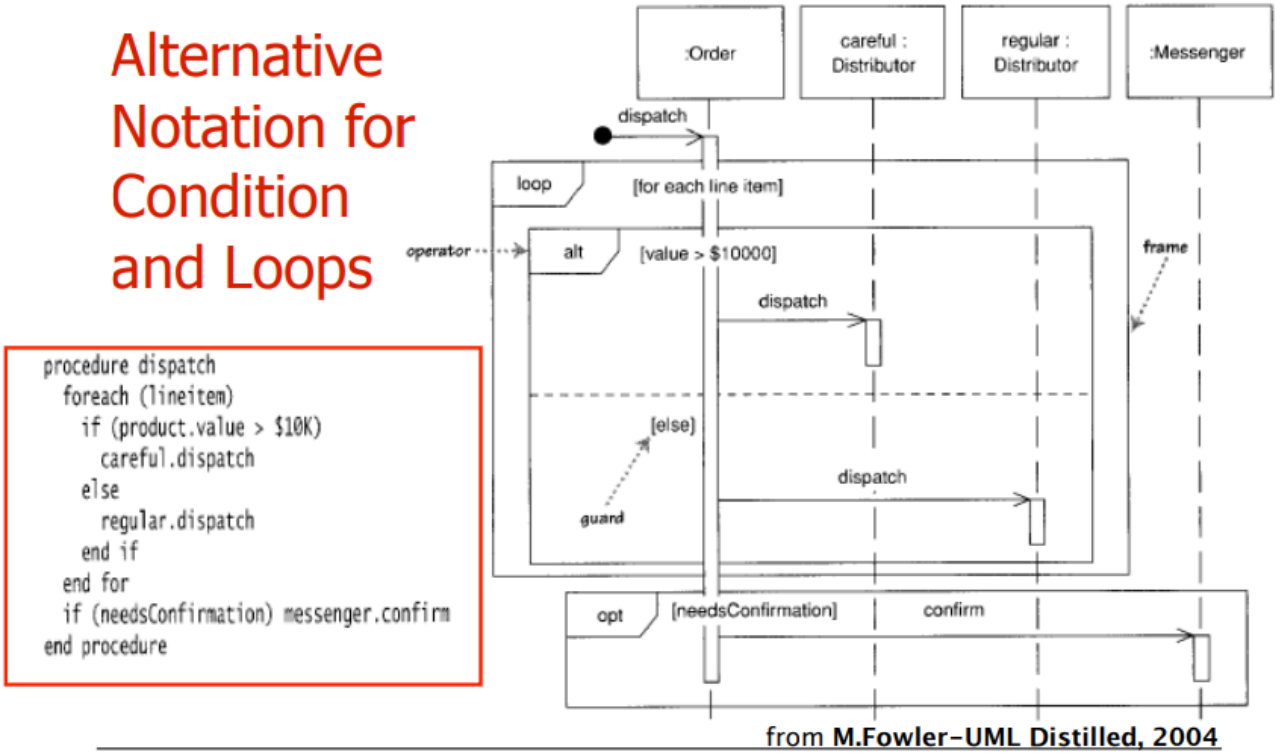
Iteration & condition

...continued from previous slide...



- Iteration is denoted by a * preceding the message name
- Condition is denoted by boolean expression in [ ] before the message name

Alternative Notation for Condition and Loops



from M.Fowler-UML Distilled, 2004

Sequence Diagram Summary

- UML sequence diagram represent behavior in terms of interactions.
- Useful to find missing objects.
- Time consuming to build but worth the investment.
- Complement the class diagrams (which represent structure).

➢ Se mapeaza ideea de scenariu pe ideea de secventa -> o diagrama de secventa se naste de obiceu dupa captarea cerintelor, dupa modelare, aplicand tehnica CRC cards si identificand relatia de obiecte